Programming Principles I

Week 4 · Class 7

Functions in C

Write it once, give it a name, call it many times

```
int square(int x) {
    return x * x;
}
```

What the loops gave us

1. A loop repeats: **start**, **check**, **body**, **update**
2. A total starts at 0, before the loop: `sum += i;`
3. A counter: `count++` inside an `if` inside the loop
4. Ask again until the input is good: `while (bad) { scanf... }`
5. A product starts at 1: `fact *= i;`

Recap: what is on the screen?

PREDICT

What does this print?

```
int count = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 4 == 0) {
        count++;
    }
}
printf("%d\n", count);
```

Recap: what is on the screen?

PREDICT

What does this print?

```
int count = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 4 == 0) {
        count++;
    }
}
printf("%d\n", count);
```

ANSWER

2

Only 4 and 8 divide by 4.

Today's roadmap

1. Why functions
2. Writing and calling a function
3. Parameters, `void` and `return`
4. Loops and decisions inside a function
5. One program, split into functions
6. Common bugs, then you write

> Every program today is in **Class 7 code** on Blackboard. You do not need to copy from the slides.

1 Why functions

Write it once. Use it many times.

Three rows of stars, copied

```
for (int i = 0; i < 3; i++) {
    printf("*");
}
printf("\n");

for (int i = 0; i < 5; i++) {
    printf("*");
}
printf("\n");

for (int i = 0; i < 7; i++) {
    printf("*");
}
printf("\n");
```

Output

```
***
*****
*******
```

The **same** loop, three times.

Only the number changes: 3, 5, 7.

What if one copy must change?

Now every row should use # instead of *.

Three places to change. Miss one, and the rows disagree.

Ten rows? Ten places.

Copied code is a bug waiting to happen. Write it **once**. Give it a **name**.

The same program, with a function

```
void print_stars(int n) {
    for (int i = 0; i < n; i++) {
        printf("*");
    }
    printf("\n");
}

int main(void) {
    print_stars(3);
    print_stars(5);
    print_stars(7);
    return 0;
}
```

Output

```
***
*****
*******
```

Written once: print_stars

```
void print_stars(int n) {
    for (int i = 0; i < n; i++) {
        printf("*");
    }
    printf("\n");
}

int main(void) {
    print_stars(3);
    print_stars(5);
    print_stars(7);
    return 0;
}
```

The loop lives here. **Once.**

n is how many stars.

Used three times

```
void print_stars(int n) {
    for (int i = 0; i < n; i++) {
        printf("*");
    }
    printf("\n");
}

int main(void) {
    print_stars(3);
    print_stars(5);
    print_stars(7);
    return 0;
}
```

Each **call** sends a different number: 3, 5, 7.

Change * to # in one place: every row changes.

You have used functions since class 1

> **`printf`, `scanf`, `sqrt`, `pow` are functions.**

Someone wrote them once, in a library.

You call them by **name**, with values in brackets: `sqrt(16)`.

Today you write your own.

2 Writing and calling a function

A name, a value in, an answer out

Our first function: square.c

```c
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main(void) {
    int y = square(5);
    printf("square(5) = %d\n", y);
    printf("square(7) = %d\n", square(7));
    return 0;
}
```

Output

```
square(5) = 25
square(7) = 49
```

Four parts of a function

Type	Name	Parameter	Body
The kind of answer `int`	How you call it `square`	The value it gets `int x`	The work, in `{ }` `return x * x;`

```
int square(int x) { return x * x; }
```

Part 1: the type of the answer

Type	Name	Parameter	Body
The kind of answer `int`	How you call it `square`	The value it gets `int x`	The work, in `{ }` `return x * x;`

`int` before the name: the answer is a whole number.

Part 2: the name

Type	Name	Parameter	Body
The kind of answer `int`	How you call it `square`	The value it gets `int x`	The work, in `{ }` `return x * x;`

You choose the name. Same rules as a variable name. Say what it does.

Part 3: the parameter

Type	Name	Parameter	Body
The kind of answer `int`	How you call it `square`	The value it gets `int x`	The work, in `{ }` `return x * x;`

A box made for the function. Every call puts a value in it.

Part 4: the body and return

Type	Name	Parameter	Body
The kind of answer `int`	How you call it `square`	The value it gets `int x`	The work, in `{ }` `return x * x;`

`return` hands the answer back to whoever called.

The call

```c
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main(void) {
    int y = square(5);
    printf("square(5) = %d\n", y);
    printf("square(7) = %d\n", square(7));
    return 0;
}
```

`square(5)`: the name, and a value in brackets.

The answer, 25, lands in `y`.

A call can sit inside printf

```c
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main(void) {
    int y = square(5);
    printf("square(5) = %d\n", y);
    printf("square(7) = %d\n", square(7));
    return 0;
}
```

`square(7)` becomes 49.

Then `printf` prints it.

Writing a function does nothing by itself

It runs only when it is called.

`int square(int x) { ... }` is the recipe.

`square(5)` is cooking it, with 5.

A program starts in `main`. A function runs only if `main` (or another function) calls it.

What happens at a call?

```
int square(int x) {
    printf("  square got %d\n", x);
    int result = x * x;
    printf("  square gives back %d\n", result);
    return result;
}

int main(void) {
    printf("main: before the call\n");
    int y = square(5);
    printf("main: got %d\n", y);
    return 0;
```

Predict: four `printf` lines. Write them down in the order they appear.

Step 1: main starts

```c
int square(int x) {
    printf("  square got %d\n", x);
    int result = x * x;
    printf("  square gives back %d\n", result);
    return result;
}

int main(void) {
    printf("main: before the call\n");
    int y = square(5);
    printf("main: got %d\n", y);
    return 0;
```

Output

```
main: before the call
```

A program always starts in `main`.

Step 2: the call jumps into square

```c
int square(int x) {
    printf("  square got %d\n", x);
    int result = x * x;
    printf("  square gives back %d\n", result);
    return result;
}

int main(void) {
    printf("main: before the call\n");
    int y = square(5);
    printf("main: got %d\n", y);
    return 0;
```

Output

```
main: before the call
```

x gets a **copy** of 5.

Step 3: the body of square runs

```c
int square(int x) {
    printf("  square got %d\n", x);
    int result = x * x;
    printf("  square gives back %d\n", result);
    return result;
}

int main(void) {
    printf("main: before the call\n");
    int y = square(5);
    printf("main: got %d\n", y);
    return 0;
```

Output

```
main: before the call
  square got 5
  square gives back 25
```

`main` waits.

Step 4: return jumps back

```c
int square(int x) {
    printf("  square got %d\n", x);
    int result = x * x;
    printf("  square gives back %d\n", result);
    return result;
}

int main(void) {
    printf("main: before the call\n");
    int y = square(5);
    printf("main: got %d\n", y);
    return 0;
```

Output

```
main: before the call
  square got 5
  square gives back 25
```

The call `square(5)` is **replaced** by 25.

Step 5: main goes on

```c
int square(int x) {
    printf("  square got %d\n", x);
    int result = x * x;
    printf("  square gives back %d\n", result);
    return result;
}

int main(void) {
    printf("main: before the call\n");
    int y = square(5);
    printf("main: got %d\n", y);
    return 0;
```

Output

```
main: before the call
  square got 5
  square gives back 25
main: got 25
```

y is 25.

A call, in four steps

1. The value in brackets is **copied** into the parameter
2. The body of the function runs; the caller waits
3. `return` sends **one** value back
4. The call is **replaced** by that value, and the caller goes on

Predict: calls inside calls

PREDICT

`square` is the same function.

What are the three lines?

```
int a = 3;
printf("%d\n", square(a) + square(4));
printf("%d\n", square(square(2)));
printf("%d\n", square(a + 1));
```

Predict: calls inside calls

PREDICT

`square` is the same function.

What are the three lines?

```
int a = 3;
printf("%d\n", square(a) + square(4));
printf("%d\n", square(square(2)));
printf("%d\n", square(a + 1));
```

ANSWER

25: 9 + 16

16: `square(2)` is 4, then `square(4)`

16: `a + 1` is 4 first

Predict: square below main

CAREFUL

The same program.

But `square` is written **below** `main`.

What happens?

```c
#include <stdio.h>

int main(void) {
    int y = square(5);
    printf("%d\n", y);
    return 0;
}

int square(int x) {
    return x * x;
}
```

gcc stops: it has not met square yet

```c
#include <stdio.h>

int main(void) {
    int y = square(5);
    printf("%d\n", y);
    return 0;
}

int square(int x) {
    return x * x;
}
```

gcc 16.2 says

```
error: implicit
declaration of
function 'square'
```

gcc reads from the **top down**.

At this line it has never heard of square.

Fix 1: the function above main

```c
#include <stdio.h>

int square(int x) {
    return x * x;
}

int main(void) {
    int y = square(5);
    printf("square(5) = %d\n", y);
    printf("square(7) = %d\n", square(7));
    return 0;
}
```

Define first. Use later.

In this course: the **whole function** above `main`.

Fix 2: a prototype above main

```c
#include <stdio.h>

int square(int x);

int main(void) {
    int y = square(5);
    printf("%d\n", y);
    return 0;
}

int square(int x) {
    return x * x;
}
```

The first line of the function, with a `;`.

It tells gcc: this function exists. Its body comes later.

3 Parameters, `void` and `return`

More values in, no answer out, an answer early

Two parameters

```c
int add(int a, int b) {
    return a + b;
}

int main(void) {
    printf("%d\n", add(2, 3));
    printf("%d\n", add(10, 20));
    return 0;
}
```

Output

```
5
30
```

Values are matched by **position**:

the first to a, the second to b.

Predict: the order of the values

PREDICT

`power(base, exp)`

What is `power(2, 3)`?

And `power(3, 2)`?

```
int power(int base, int exp) {
    int result = 1;
    for (int i = 1; i <= exp; i++) {
        result *= base;
    }
    return result;
}
```

The order matters

```c
int power(int base, int exp) {
    int result = 1;
    for (int i = 1; i <= exp; i++) {
        result *= base;
    }
    return result;
}

int main(void) {
    printf("power(2, 3) = %d\n", power(2, 3));
    printf("power(3, 2) = %d\n", power(3, 2));
    printf("power(10, 0) = %d\n", power(10, 0));
```

Output

```
power(2, 3) = 8
power(3, 2) = 9
power(10, 0) = 1
```

$2 \times 2 \times 2 = 8$

$3 \times 3 = 9$

A loop inside a function

`power` has its own loop and its own `result`.

`main` does not see them. It only sees the answer.

`power(10, 0)` is 1: the loop runs zero times.

Predict: the average of 7 and 8

PREDICT

Two functions.

Both get 7 and 8.

What does each give back?

```
int average_int(int a, int b) {
    return (a + b) / 2;
}

double average(double a, double b) {
    return (a + b) / 2;
}
```

int or double: integer division again

```c
int average_int(int a, int b) {
    return (a + b) / 2;
}

double average(double a, double b) {
    return (a + b) / 2;
}

int main(void) {
    printf("int version: %d\n", average_int(7, 8));
    printf("double version: %.2f\n", average(7, 8));
    return 0;
```

Output

```
int version: 7
double version: 7.50
```

`int`: 15 / 2 is 7. The .5 is gone.

`double`: 7.50.

void: a function with no answer

```
void print_stars(int n)
```

`void` means: it gives **nothing** back.

It does a job, like printing. `return` is not needed.

void with no parameters

```c
#include <stdio.h>

void greet(void) {
    printf("Hello!\n");
    printf("Welcome to CSCI 1101.\n");
}

int main(void) {
    greet();
    greet();
    return 0;
}
```

Output

```
Hello!
Welcome to CSCI 1101.
Hello!
Welcome to CSCI 1101.
```

`(void)`: it takes nothing either.

The call still needs brackets: `greet();`

Printing is not returning

PREDICT

One function **prints** the square.

One **returns** it.

What is on the screen?

```
void square_print(int x) {
    printf("%d\n", x * x);
}

int square(int x) {
    return x * x;
}

int main(void) {
    square_print(5);
    int total = square(3) + square(4);
    printf("total = %d\n", total);
    return 0;
```

Printing is not returning

```c
void square_print(int x) {
    printf("%d\n", x * x);
}

int square(int x) {
    return x * x;
}

int main(void) {
    square_print(5);
    int total = square(3) + square(4);
    printf("total = %d\n", total);
    return 0;
```

Output

```
25
total = 25
```

`square_print(5)` prints 25 by itself.

`square` hands 9 and 16 back, so `main` can add them.

You cannot add two printouts

```c
void square_print(int x) {
    printf("%d\n", x * x);
}

int main(void) {
    int total = square_print(3) + square_print(4);
    printf("total = %d\n", total);
    return 0;
}
```

gcc 16.2 says

```
error: void value not ignored as it ought to be
```

A `void` function gives back nothing.

Print or return?

Print	Return
Shows it on the screen. The caller gets **nothing**.	Hands the value back. The caller can **use** it.

> Most functions should **return**. Let `main` decide what to print.

return ends the function at once

```
int max(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
```

If `a > b`: `return a;`

The function **ends** here.
The rest is skipped.

Otherwise: the second return

```
int max(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
```

Output

```
max(3, 9) = 9
max(9, 3) = 9
```

We only get here when a > b was false.

Predict: a function that calls a function

PREDICT

`max3` calls `max` twice.

What is `max3(4, 11, 7)`?

```
int max3(int a, int b, int c) {
    return max(max(a, b), c);
}
```

Predict: a function that calls a function

PREDICT

`max3` calls `max` twice.

What is `max3(4, 11, 7)`?

```
int max3(int a, int b, int c) {
    return max(max(a, b), c);
}
```

ANSWER

11

`max(4, 11)` is 11.

Then `max(11, 7)` is 11.

Yes or no: return 1 or 0

```
int is_even(int n) {
    if (n % 2 == 0) {
        return 1;
    }
    return 0;
}
```

It gives back **1** for yes, **0** for no.

In C, 1 is true and 0 is false.

The call goes straight into an if

```
for (int i = 1; i <= 5; i++) {
    if (is_even(i)) {
        printf("%d is even\n", i);
    } else {
        printf("%d is odd\n", i);
    }
}
```

Output

```
1 is odd
2 is even
3 is odd
4 is even
5 is odd
```

`if (is_even(i))` reads like English.

4

Loops and decisions inside a function

Class 4 and class 6, now with names

Factorial as a function

```c
int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}

int main(void) {
    for (int n = 0; n <= 10; n++) {
        printf("%d! = %d\n", n, factorial(n));
    }
```

Output

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

Why is 0! equal to 1?

QUESTION

Call `factorial(0)`.

How many times does the loop run?

So what comes back?

```
int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Why is 0! equal to 1?

QUESTION

Call `factorial(0)`.

How many times does the loop run?

So what comes back?

```
int factorial(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

ANSWER

Zero times: `2 <= 0` is false. `result` is still 1.

A letter grade: char letter(int score)

```
char letter(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 80) {
        return 'B';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'D';
    }
    return 'F';
}
```

It returns a `char`: one letter.

Inside: the else-if chain of class 4.

Using letter

```
int main(void) {
    printf("95 -> %c\n", letter(95));
    printf("72 -> %c\n", letter(72));
    printf("40 -> %c\n", letter(40));
    return 0;
}
```

Output

```
95 -> A
72 -> C
40 -> F
```

> %c prints one char.

The input check of class 6, as a function

```
int read_between(int low, int high) {
    int value;
    scanf("%d", &value);
    while (value < low || value > high) {
        printf("Type %d to %d: ", low, high);
        scanf("%d", &value);
    }
    return value;
}
```

It keeps asking until the value is between `low` and `high`.

Then it **returns** it.

One function, two different checks

```c
int main(void) {
    printf("Month: ");
    int month = read_between(1, 12);
    printf("Day: ");
    int day = read_between(1, 31);
    printf("Date: %d/%d\n", day, month);
    return 0;
```

Output

```
Month: 13
Type 1 to 12: 0
Type 1 to 12: 7
Day: 40
Type 1 to 31: 25
Date: 25/7
```

5 One program, split into functions

`main` reads like a list of steps

The job

> **For three students: read a score, and print the grade.**

Read a score from 0 to 100. Ask again if it is bad.

Work out the letter: A, B, C, D or F.

Print one report line.

You could write it all in main

It works. It is 27 lines in one block.

Reading, checking, grading and printing are mixed together.

It is in the class code: `21_report_all_in_main.c`.

To change the grading, you must first **find** it.

Split: main is a list of steps

```c
int main(void) {
    for (int i = 1; i <= 3; i++) {
        printf("Score of student %d: ", i);
        int score = read_between(0, 100);
        print_report(i, score);
    }
    return 0;
}
```

Read a score.

Print the report.

Each step has a **name**.

Step 1: read_between

```
int read_between(int low, int high) {
    int value;
    scanf("%d", &value);
    while (value < low || value > high) {
        printf("Type %d to %d: ", low, high);
        scanf("%d", &value);
    }
    return value;
}
```

> From two slides ago. Copy it in.

Step 2: letter

```
char letter(int score) {
    if (score >= 90) {
        return 'A';
    } else if (score >= 80) {
        return 'B';
    } else if (score >= 70) {
        return 'C';
    } else if (score >= 60) {
        return 'D';
    }
    return 'F';
}
```

From `18_letter.c`. Copy it in.

Step 3: print_report

```
void print_report(int n, int score) {
    printf("Student %d: %d points, ", n, score);
    printf("grade %c\n", letter(score));
}
```

A function can call another function: `letter(score)`.

Run it

```
int main(void) {
    for (int i = 1; i <= 3; i++) {
        printf("Score of student %d: ", i);
        int score = read_between(0, 100);
        print_report(i, score);
    }
    return 0;
}
```

Output

```
Score of student 1: 95
Student 1: 95 points, grade A
Score of student 2: 72
Student 2: 72 points, grade C
Score of student 3: 40
Student 3: 40 points, grade F
```

A bad score is caught inside read_between

```c
int main(void) {
    for (int i = 1; i <= 3; i++) {
        printf("Score of student %d: ", i);
        int score = read_between(0, 100);
        print_report(i, score);
    }
    return 0;
}
```

Output

```
Score of student 1: 120
Type 0 to 100: 88
Student 1: 88 points, grade B
Score of student 2: 60
Student 2: 60 points, grade D
Score of student 3: 59
Student 3: 59 points, grade F
```

Why split a program?

Read	Change	Test	Reuse
`main` reads like a list of steps.	New grades? Change one function.	Try `letter(89)` on its own.	`read_between` works for months, days, scores.

6 Common bugs

Each one compiled with gcc 16.2

Five function bugs

1. A missing `return`
2. The wrong number of values in a call
3. A `;` after the first line of the function
4. A call without `( )`
5. A function used before gcc has seen it

Predict: one return is missing

```
int max(int a, int b) {
    if (a > b) {
        return a;
    }
}

int main(void) {
    printf("max(9, 3) = %d\n", max(9, 3));
    printf("max(3, 9) = %d\n", max(3, 9));
    return 0;
```

There is no `return` when `a > b` is false.

Predict: what does `max(3, 9)` print?

A missing return: a warning, then a wrong answer

```c
int max(int a, int b) {
    if (a > b) {
        return a;
    }
}

int main(void) {
    printf("max(9, 3) = %d\n", max(9, 3));
    printf("max(3, 9) = %d\n", max(3, 9));
    return 0;
```

Output

```
max(9, 3) = 9
max(3, 9) = 3
```

gcc: `control reaches end of non-void function`

It builds. It runs. It is **wrong**.

The wrong number of values

```
printf("%d\n", square(3, 4));
printf("%d\n", add(5));
```

gcc 16.2 says

```
error: too many arguments
to function 'square';
expected 1, have 2

error: too few arguments
to function 'add';
expected 2, have 1
```

A semicolon after the first line

```
int square(int x);
{
    return x * x;
}
```

gcc 16.2 says

```
error: expected
identifier or '('
before '{' token
```

The `;` makes it a prototype.

The `{` after it belongs to nothing.

A call without brackets

```c
int main(void) {
    greet();
    greet;
    printf("end\n");
    return 0;
}
```

Output

```
Hello!
end
```

gcc -Wall: `statement with no effect`

`greet;` names it.
`greet();` calls it.

What gcc says

Bug	gcc 16.2	Program built?
no `return`	warning: control reaches end...	**built**
wrong number of values	error: too many / too few arguments	none
`;` after the first line	error: expected identifier or '('	none
no `()` in the call	warning: statement with no effect	**built**
used before it is seen	error: implicit declaration	none

The two **built** rows are the dangerous ones: a program that runs wrong. Always use `-Wall` and read it.

7

Now you write

Two functions in the room, two at home

Four small functions

1. **Task 1:** `int cube(int x)`
2. **Task 2:** `int sign(int n)`: -1, 0 or 1
3. Task 3: `void print_rectangle(int width, int height)`
4. Task 4: `double celsius_to_fahrenheit(double c)`

> Tasks 1 and 2 now. 3 and 4 at home. Write the function **above** `main`, then call it.

Task 1: cube

YOUR TURN

Write `int cube(int x)`.

In `main`, read a number and print its cube.

```
Number: 3
Cube: 27
```

Task 2: sign

YOUR TURN

Write `int sign(int n)`.

It gives back -1 if `n` is negative, 0 if it is zero, and 1 if it is positive.

```
Number: -7
Sign: -1
```

Solutions: cube and sign

Task 1

```
int cube(int x) {
    return x * x * x;
}
```

Task 2

```
int sign(int n) {
    if (n < 0) {
        return -1;
    } else if (n == 0) {
        return 0;
    }
    return 1;
}
```

Yours may look different and still be right. Test it with a negative number and with 0.

Task 3, at home: a rectangle

YOUR TURN

Write the function `print_rectangle`.

It gets a width and a height.

Hint: call `print_stars` once for every row.

`print_rectangle(6, 3)` prints:

```
******
******
******
```

Task 4, at home: Celsius to Fahrenheit

YOUR TURN

Write the function `celsius_to_fahrenheit`.

It gets a `double`, gives back a `double`.

The rule: c × 9 / 5 + 32.

```
Celsius: 100
Fahrenheit: 212.0
```

Class 7 summary

1. A function: **type**, **name**, **parameters**, **body**
2. A call copies the values in and gets **one** value back
3. Write the function **above** `main` (or give a prototype)
4. `void`: no answer. It only does a job.
5. `return` ends the function at once
6. Split a program so `main` reads like a list of steps

Project 1 is out

Simplified Blackjack: your first whole program.

Loops, decisions, and **functions** from today.

The brief is on Blackboard. Due **Sun 11 October, 23:59**.

Start this week. A game takes time to test.

Before Friday

1. **Knowledge check 7** on Blackboard: today's slides
2. Tasks **3 and 4**: bring them, working or not
3. **Quiz 2: Fri 2 October**, first 25 minutes, classes 3 to 6
4. Today's code: Blackboard, Week 4 module, **Class 7 code**

> Friday, after the quiz: where variables live, why a function cannot change yours, and a function that calls itself.